

Titulo: Velgrym

Autor: José Antonio Villodres Zafra

Curso: 2ºDAM-2024-2025

Fecha de entrega: por determinar

Índice

1. Portada
2. Índice
3. Introducción del proyecto
4. Objetivos del Proyecto
 - 4.1. Objetivo general
 - 4.2. Objetivos específicos
5. Descripción del Proyecto
 - 5.1. Resumen general del videojuego
 - 5.2. Historia y ambientación
 - 5.3. Mecánicas principales
 - 5.4. Personajes jugables
 - 5.5. Enemigos y obstáculos
 - 5.6. Diseño visual y sonoro
6. Tecnologías utilizadas
 - 6.1. Godot Engine
 - 6.2. Herramientas de diseño gráfico
 - 6.3. Recursos de audio y música
 - 6.4. Control de versiones y documentación
7. Metodología de trabajo
 - 7.1. Modelo de desarrollo elegido
 - 7.2. Justificación de la metodología
 - 7.3. Adaptaciones durante el proceso
8. Planificación y cronograma
 - 8.1. Fases del desarrollo
 - 8.2. Cronograma estimado
 - 8.3. Ajustes sobre la marcha
9. Desarrollo del Proyecto
 - 9.1. Preparación del entorno y organización inicial
 - 9.2. Sistema de movimiento y físicas básicas
 - 9.3. Personajes jugables: implementación modular
 - 9.4. Sistema de combate, vida y muerte
 - 9.5. Inteligencia artificial y comportamiento de enemigos
 - 9.6. Creación de mapas y elementos interactivos
 - 9.7. HUD, cámara y transiciones de escenas
 - 9.8. Pruebas, ajustes y resolución de errores
 - 9.9. Documentación y control de calidad final
10. Resultados
11. Conclusiones y Futuras Mejoras
 - 11.1. Valoración personal del proyecto
 - 11.2. Propuestas de mejora
 - 11.3. Líneas futuras de desarrollo
12. Bibliografía
13. Anexos

3. Introducción del proyecto:

Velgrym es un videojuego 2D tipo metroidvania, desarrollado en el motor Godot, con una estética pixel art y ambientación inspirada en la fantasía oscura. El proyecto combina exploración, combate en tiempo real y progresión mediante personajes con habilidades únicas, dentro de un mundo interconectado que favorece la rejugabilidad.

Este trabajo surge como proyecto final del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma, con el objetivo de aplicar conocimientos técnicos adquiridos durante el curso en un entorno práctico y creativo. La creación de un videojuego permite integrar áreas como el diseño modular, la programación orientada a objetos, la inteligencia artificial básica, la gestión de escenas, el diseño visual y la documentación técnica.

Hasta el momento se ha implementado una estructura jugable sólida: dos personajes funcionales con estilos diferentes (el Caballero y el Gato), un sistema modular de combate y vida reutilizable, varios enemigos con comportamientos distintos, dos mapas conectados, HUD operativo, cámara dinámica, elementos interactivos y control de versiones con Git.

Además de consolidar una base técnica estable, el proyecto está preparado para crecer. Entre las funcionalidades planificadas se encuentran la incorporación de un tercer personaje (el Druida), nuevos enemigos y jefes, más zonas del mapa, un sistema de guardado y, como mecánica clave, la posibilidad de acceder a un “mundo espejo” que amplíe la experiencia de exploración.

Velgrym ha sido concebido no solo como una aplicación académica, sino como un entorno donde experimentar libremente con ideas de diseño, narrativa y jugabilidad, aplicando una metodología ágil que ha permitido iterar y mejorar el juego de forma progresiva.

4. Objetivos del Proyecto

4.1. Objetivo general

El objetivo principal del proyecto es desarrollar un videojuego multiplataforma en 2D utilizando el motor Godot, que combine mecánicas de plataformas, combate y exploración dentro de una estructura no lineal inspirada en el género metroidvania. Este proyecto se enmarca en el Trabajo de Final de Ciclo de 2º DAM y tiene como finalidad integrar de forma práctica los conocimientos adquiridos en programación, diseño de videojuegos, estructuración de código, uso de control de versiones y desarrollo ágil.

El juego debe ser funcional, con una base técnica sólida que permita una posible ampliación posterior, y que represente fielmente el trabajo realizado durante el curso tanto en su dimensión técnica como creativa.

4.2. Objetivos específicos

A lo largo del desarrollo, se han establecido los siguientes objetivos concretos, de los cuales una parte ya ha sido completada satisfactoriamente:

- Diseñar un sistema de niveles interconectados, con mapas explorables que permitan rutas alternativas y zonas bloqueadas que se desbloquean en función del personaje o las habilidades adquiridas.
- Implementar un sistema de movimiento y físicas adaptado a personajes con estilos diferentes, incluyendo salto, caída, detección de suelo y mecánicas adicionales como escalada o salto en pared.
- Crear al menos dos personajes jugables (Caballero y Gato), cada uno con habilidades diferenciadas: el primero centrado en el combate cuerpo a cuerpo, y el segundo enfocado en movilidad y exploración.
- Desarrollar enemigos con comportamientos diversos utilizando inteligencia artificial básica, incluyendo persecución, patrullaje, ataques y retorno a posición inicial.

- Diseñar un sistema modular de combate, vida y daño que sea reutilizable para personajes y enemigos, incluyendo retroceso, invulnerabilidad temporal y animaciones de impacto o muerte.
- Construir dos mapas funcionales (cueva de inicio y pradera), conectados entre sí, que sirvan como entorno de prueba para las principales mecánicas del juego.
- Implementar una interfaz de usuario básica con HUD funcional (barra de vida), cámara dinámica asociada al personaje y transiciones entre escenas jugables.
- Utilizar Git y GitHub para el control de versiones, con ramas diferenciadas por módulos y documentación de los cambios mediante commits descriptivos.
- Documentar el desarrollo técnico, la evolución del proyecto y las decisiones de diseño en una memoria clara, estructurada y adaptada al entorno académico.

Objetivos planificados para versiones futuras

- Aunque el juego ya dispone de una base funcional sólida, se han definido objetivos adicionales que se implementarán en futuras fases del proyecto o tras su entrega académica:
- Añadir el tercer personaje jugable: el Druida, con habilidades mágicas y mecánicas de apoyo a distancia.
- Ampliar el mundo del juego con nuevos mapas conectados, zonas ocultas y elementos interactivos adicionales como puertas mágicas, trampas, objetos coleccionables o caminos alternativos según el personaje.
- Diseñar enemigos más complejos, con mecánicas únicas y posibles jefes (bosses) que representen un desafío superior.
- Implementar un sistema de guardado manual a través de puntos de control repartidos por el mapa, incluyendo restauración de salud y acceso a portales de viaje rápido.
- Incluir una mecánica avanzada de exploración mediante el mundo espejo, una dimensión paralela con sus propias reglas, enemigos y rutas, que amplíe la profundidad del juego sin necesidad de crear contenido completamente nuevo desde cero.
- Mejorar el HUD con indicadores adicionales: objetos, habilidades activas, mapa o progreso general.
- Incorporar narrativa ligera mediante diálogos opcionales o textos repartidos por el mundo que amplíen el contexto del juego sin interrumpir la jugabilidad.

- También se contempla la incorporación de nuevas pistas musicales originales para cada zona del juego, con el objetivo de reforzar la ambientación y mejorar la inmersión del jugador.

5. Descripción del Proyecto

5.1. Resumen general del videojuego

Velgrym es un videojuego 2D del género metroidvania, desarrollado con el motor Godot. Está diseñado para ejecutarse en ordenadores, con posibilidad de adaptarse a otras plataformas. Combina mecánicas clásicas de plataformas con combate en tiempo real, exploración libre y progresión mediante habilidades y personajes diferenciados.

El estilo visual se basa en pixel art con una ambientación de fantasía oscura, inspirada en títulos retro, pero con un enfoque más dinámico y colorido. Actualmente, el juego cuenta con dos personajes jugables, enemigos con distintos comportamientos, un sistema de combate modular y dos mapas conectados. El diseño modular y el uso de control de versiones han permitido construir una base estable y escalable.

5.2. Historia y ambientación

Hace siglos, una guerra devastadora enfrentó a los humanos contra las criaturas míticas más feroces de la tierra media, Los dragones, guardianes ancestrales de la magia. El conflicto dejó todo el mundo fragmentado y en ruinas, donde la magia se volvió arcana y salvaje mientras que las naciones se fragmentan y toman bandos, sumidas en el caos.

Con el tiempo, los dragones desaparecieron escondiéndose en torreones abandonados o cuevas remotas debido a la constante persecución que se les lleva haciendo durante siglos.

La humanidad, creyéndose vencedora, levantó reinos sobre las cenizas del conflicto olvidando poco a poco las consecuencias de su ambición.

Pero la amenaza nunca desapareció, en secreto un anciano arcano exiliado ha logrado dominar fragmentos de la magia salvaje que quedó tras la guerra. Obsesionado con el poder ancestral de los dragones, ha iniciado rituales antiguos con el fin de despertarlos, y controlarlos. Para ello él y su ejército de no muertos atacan el reino para reunir las piezas mágicas de Velgrym que están repartidas por el mundo.

Su plan no ha pasado desapercibido. A medida que el ejército del arcano avanza, los cielos se oscurecen y las tierras tiemblan bajo su paso. Las piezas de Velgrym, cuatro reliquias diacrónicas cargadas de una energía ancestral, comienzan a resonar con una fuerza que llevaba siglos dormida. Se dice que, unidas, pueden abrir el umbral que separa el mundo actual del dominio perdido de los dragones.

Hasta ahora, el arcano solo ha logrado apoderarse de una de ellas. Las otras tres permanecen ocultas en distintas regiones del mundo, protegidas por antiguos lugares místicos y entornos salvajes transformados por la magia caótica de los fragmentos.

Ante esta amenaza, surgen tres viajeros inesperados, cada uno con sus propios motivos, pero unidos por un mismo destino: detener al arcano antes de que reúna los fragmentos restantes. Un caballero errante marcado por las historias de las antiguas guerras, un druida guardián de secretos antiguos, y un enigmático y divertido gato con habilidades fuera de lo común el cual le parece muy interesante lo brillante. Juntos deberán recorrer tierras corrompidas, enfrentarse a criaturas surgidas de la magia salvaje u esconderse tras la maleza para evitar conflictos. Todo ello para descubrir la verdad que esconde tras la leyenda de Velgrym.

5.3. Mecánicas principales

El núcleo jugable de Velgrym se basa en ofrecer una experiencia variada, accesible y expandible:

- Exploración no lineal: el mapa está compuesto por zonas interconectadas que no siguen una estructura lineal. El acceso a nuevas áreas depende del personaje y de las habilidades desbloqueadas.
- Combate en tiempo real: cada personaje presenta un enfoque distinto. El Caballero lucha cuerpo a cuerpo con retroceso; el Gato esquiva y aturde enemigos. El Druida (aún no implementado) usará magia a distancia.
- Progresión basada en habilidades: se incorporan habilidades como el doble salto, deslizamiento por paredes o escalada temporal, que permiten descubrir rutas alternativas.
- Entorno interactivo: se han incluido elementos como plataformas móviles, trampas, puertas mágicas y zonas frágiles, algunos de los cuales solo pueden activarse o evitarse con un personaje específico.

- Sistema de puntos de guardado (pendiente de implementación): el juego no guarda automáticamente, pero se planea un sistema de guardado manual en zonas concretas del mapa.

5.4. Personajes jugables

El jugador puede elegir entre tres personajes con estilos y mecánicas distintas:

- **Caballero:** personaje equilibrado, orientado al combate cuerpo a cuerpo. Utiliza una maza pesada que afecta a su velocidad y lo empuja ligeramente al atacar. Es ideal para avanzar de forma frontal, golpeando a los enemigos directamente.
- **Gato:** personaje rápido y ágil. No puede atacar directamente, pero puede aturdir enemigos, moverse con mayor velocidad, pasar por huecos estrechos de 1x1 bloques y escalar paredes durante un tiempo limitado. Es perfecto para evitar combates y explorar zonas elevadas o secretas.
- **Druida** (Por implementar): se trata de un personaje mágico que usará hechizos naturales a distancia, habilidades de curación y posible manipulación del entorno. Su estilo de juego estará centrado en el control del espacio y el apoyo.

Cada personaje ofrece una forma distinta de recorrer el mundo de Velgrym, lo que favorece la rejugabilidad y permite acceder a zonas exclusivas según sus habilidades.

5.5. Enemigos y obstáculos

Durante la aventura, el jugador se enfrentará a distintos enemigos repartidos por las zonas del mapa. Cada uno presenta un comportamiento específico que obliga a adaptar la estrategia según el personaje controlado. Hasta la fecha se han implementado cinco tipos de enemigos funcionales:

- **Slime:** enemigo básico que se desliza por el suelo y puede trepar por paredes durante un tiempo limitado. Persigue al jugador al detectarlo.
- **Murciélago:** permanece colgado en el techo y se lanza en picado al detectar al jugador. Si falla, regresa a su posición original.

- Seta: camina de un lado a otro sin detectar al jugador. Solo puede ser derrotada si se le cae encima, lo que obliga a usar el entorno.
- Planta lanzaguisantes: permanece estática y dispara proyectiles a intervalos. Añade dificultad al desplazamiento, especialmente en zonas abiertas.
- Fantasma: flota por el escenario atravesando paredes y sigue al jugador lentamente. No se puede derrotar fácilmente y actúa como obstáculo persistente.

Estos enemigos están distribuidos estratégicamente en el mapa y presentan diferentes niveles de amenaza según el personaje seleccionado. Por ejemplo, el Caballero puede enfrentarse directamente a la mayoría de ellos, mientras que el Gato debe esquivar, explorar rutas alternativas o usar su agilidad para evitarlos.

Además, el mundo de Velgrym incluye elementos interactivos como:

- Plataformas móviles que permiten acceder a zonas elevadas.
- Huecos estrechos diseñados para el Gato.
- Zonas verticales que requieren habilidades como escalada o doble salto.

Actualmente no se han implementado trampas, puertas mágicas, rocas rompibles ni mecánicas avanzadas de entorno, pero están previstas para futuras versiones. Estos elementos servirán para enriquecer la exploración y aumentar el reto del jugador.

También se contempla la incorporación de jefes principales con mecánicas únicas, fases de combate y escenarios propios. Estos encuentros actuarán como hitos dentro del progreso del juego, ofreciendo desafíos mayores y desbloqueando nuevas áreas tras su derrota.

5.6. Diseño visual y sonoro

El diseño visual y sonoro de Velgrym está planificado como un componente clave para reforzar la ambientación del juego, pero todavía no se ha implementado más allá de pruebas básicas o recursos preparados para futuras versiones.

El estilo visual estará basado en pixel art, utilizando assets gratuitos adaptados con herramientas como itch.io, GIMP o Piskel. Se busca una estética coherente que combine el tono oscuro con colores vivos según la zona del mapa.

A nivel sonoro, se prevé incluir música ambiental para cada zona, efectos simples (saltos, ataques, daño) y sonidos ambientales. Todo ello se integrará en versiones posteriores, con el objetivo de reforzar la inmersión y dar identidad sonora al mundo de Velgrym.

El diseño artístico y de audio se desarrollará una vez estén consolidadas las mecánicas, enemigos y niveles, para asegurar una implementación acorde al estilo jugable del proyecto.

6. Tecnologías utilizadas

Durante el desarrollo de Velgrym se han empleado diversas tecnologías tanto para la programación como para el diseño gráfico y la organización del proyecto. La elección de cada herramienta ha estado basada en criterios como accesibilidad, compatibilidad con el motor principal (Godot), facilidad de uso y disponibilidad gratuita o libre para uso académico.

6.1. Godot Engine

El motor utilizado para desarrollar el videojuego ha sido Godot Engine, una plataforma de desarrollo libre y de código abierto, ampliamente utilizada en entornos educativos y por desarrolladores independientes. Su sistema de nodos, escenas y scripts permite una estructura clara, modular y fácilmente escalable.

Se ha trabajado con GDScript, el lenguaje nativo de Godot, que presenta una sintaxis similar a Python. Esto ha facilitado la implementación de mecánicas como el sistema de movimiento, combate, IA enemiga, vida y daño, así como la gestión de escenas, HUD, cámara y transiciones.

Entre las ventajas más destacadas de Godot en este proyecto están:

- Curva de aprendizaje asequible.
- Herramientas integradas para animación, físicas, señales y colisiones.
- Sistema de herencia entre nodos para reutilizar código en personajes y enemigos.

- Depuración visual y consola integrada.

Toda la estructura del juego está dividida en escenas reutilizables, organizadas por nodos con scripts independientes, siguiendo buenas prácticas de modularidad.

6.2. Herramientas de diseño gráfico

Para el apartado visual del proyecto Velgrym, se ha optado por utilizar recursos gráficos prediseñados (assets) provenientes de plataformas de distribución gratuita. Esta decisión ha permitido centrar el desarrollo en la programación y la lógica de juego, sin comprometer la coherencia visual general. Esta estrategia es habitual tanto en proyectos educativos como en producciones independientes con recursos limitados.

Los assets han sido seleccionados por su compatibilidad con Godot, su estilo coherente en pixel art y sus licencias libres para uso académico o comercial. Aunque no se ha trabajado todavía en un sistema visual definitivo o personalizado, estos recursos han servido como base para los personajes jugables, enemigos y escenarios ya funcionales. A continuación, se detallan las fuentes principales utilizadas:

- ToffeeCraft – Cat Pack (<https://toffeecraft.itch.io/cat-pack>)

Paquete de animaciones felinas en pixel art. Ha sido utilizado como base visual para el personaje jugable del Gato.

- Game Endeavor – Mystic Woods (<https://game-endeavor.itch.io/mystic-woods>)

Incluye tilesets naturales, árboles, rocas y estructuras rústicas. Ha sido esencial para el diseño del mapa de pradera.

- PixelFrog – Treasure Hunters & Pixel Adventure 1
 - (<https://pixelfrog-assets.itch.io/treasure-hunters>)
 - (<https://pixelfrog-assets.itch.io/pixel-adventure-1>)

Contienen enemigos, cofres, plataformas móviles y decoraciones. Se han usado para poblar el entorno y definir obstáculos y referencias visuales en diferentes zonas.

- CraftPix – Free Slime Sprite Sheets (Pixel Art) (<https://craftpix.net/freebies/free-slime-sprite-sheets-pixel-art/>)

Sprite sheets animados utilizados para el enemigo tipo slime, con distintas expresiones y estados.

En cuanto a herramientas de edición, se ha utilizado la versión gratuita de Aseprite para recolorear sprites, dividir hojas de animación y realizar pequeños ajustes visuales. También se han empleado herramientas como Piskel y GIMP para ediciones puntuales o conversiones rápidas, especialmente en ajustes de tamaño y transparencias.

Hasta el momento, no se ha desarrollado ningún recurso gráfico propio ni se ha personalizado la interfaz visual (menús, botones o cinemáticas), pero se contempla incluir mejoras en fases posteriores del desarrollo, especialmente a medida que se definan más zonas, jefes y elementos interactivos del mapa. La creación de una identidad visual única será parte del trabajo pendiente en futuras versiones.

Esta combinación de reutilización y edición básica ha permitido construir un entorno visual funcional y coherente, adecuado para probar y demostrar las mecánicas implementadas sin desviar el foco del desarrollo técnico principal.

6.3. Recursos de audio y música

El apartado sonoro de Velgrym aún no ha sido implementado, pero está previsto para fases futuras del desarrollo. El objetivo será acompañar la jugabilidad con efectos básicos (saltos, ataques, daño, muerte) y música ambiental que refuerce la atmósfera de cada zona sin resultar intrusiva.

Se han preseleccionado recursos gratuitos con licencia libre para su futura integración:

- Kenney.nl: efectos de sonido estilo retro (saltos, golpes, menús).

- [Freesound.org](https://freesound.org): sonidos ambientales (ecos, crujidos, magia).
- [OpenGameArt.org](https://opengameart.org) / [FreePD.com](https://freepd.com): música ambiental libre y adaptable.

En versiones posteriores se prevé la incorporación de pistas musicales personalizadas por mapa, efectos contextuales y sonidos que acompañen acciones clave. El diseño sonoro se abordará una vez estén consolidadas las mecánicas principales y la estructura del juego.

6.4. Control de versiones y documentación

Durante el desarrollo de Velgrym se ha utilizado el sistema de control de versiones Git, junto con un repositorio remoto privado en GitHub, como herramienta principal para gestionar el código fuente del proyecto. Esta metodología ha permitido mantener un flujo de trabajo organizado, registrar cambios de forma segura y evitar pérdidas de información.

Aunque no se ha trabajado con múltiples ramas, todo el desarrollo se ha llevado a cabo de manera continua sobre una rama principal única, realizando commit frecuentes y descriptivos que reflejan con claridad cada avance, corrección o implementación. Esta práctica ha facilitado la posibilidad de retroceder ante errores, mantener versiones estables y documentar el progreso paso a paso.

La estructura del repositorio ha estado organizada por carpetas temáticas dentro del proyecto de Godot, separando personajes, enemigos, sistemas, mapas, HUD y recursos. Esta organización interna ha sido clave para garantizar la modularidad del código y la claridad del entorno de trabajo, especialmente conforme el número de escenas y scripts ha ido creciendo.

La documentación del proyecto se ha gestionado a tres niveles:

- **Memoria técnica:** este documento, redactado de forma paralela al desarrollo, que detalla el proceso seguido, las decisiones de diseño, las dificultades técnicas encontradas y las soluciones aplicadas.
- **Comentarios internos en el código:** los scripts más relevantes cuentan con anotaciones que explican su funcionalidad, parámetros clave y posibles puntos de ampliación. Esto favorece la comprensión del código y su mantenibilidad a largo plazo.

- **Jerarquía estructurada dentro del proyecto Godot:** cada elemento del juego (personajes, enemigos, niveles, sistemas) se encuentra organizado en carpetas independientes, lo que facilita su localización, reutilización y expansión.

A pesar de tratarse de un desarrollo individual, se ha mantenido una metodología cercana a un entorno profesional, aplicando prácticas reales de control de versiones, documentación y modularización del proyecto. Este enfoque ha permitido desarrollar nuevas funcionalidades sin comprometer la estabilidad general del juego y ha proporcionado un marco de trabajo claro y mantenible.

7. Metodología de trabajo

7.1. Modelo de desarrollo elegido

Para el desarrollo de Velgrym se ha seguido una metodología ágil, inspirada en el modelo Scrum, adaptada a un entorno individual. Aunque no se han utilizado herramientas formales de gestión de equipos, sí se ha trabajado por sprints semanales, con objetivos concretos definidos al inicio de cada etapa.

Este enfoque ha permitido mantener un ritmo de trabajo constante, centrado en entregas funcionales frecuentes, con margen para reorganizar prioridades o realizar cambios sin afectar negativamente al progreso general. El proyecto se ha desarrollado de forma incremental, aplicando mejoras de forma continua y validando cada sistema antes de integrar el siguiente.

7.2. Justificación de la metodología

La elección de un enfoque ágil se justifica por la naturaleza dinámica del proyecto. Al tratarse de un videojuego con múltiples elementos interconectados (personajes, enemigos, movimiento, niveles, HUD...), no era viable prever todos los detalles desde el inicio.

Trabajar en sprints ha permitido:

- Dividir el proyecto en módulos independientes y funcionales.
- Aplicar mejoras progresivas en sistemas ya implementados.

- Detectar errores en fases tempranas.
- Ajustar mecánicas jugables según el resultado de las pruebas.
- Mantener la motivación con entregas jugables frecuentes.

Esta metodología ha sido especialmente útil en un desarrollo individual, permitiendo tomar decisiones flexibles y adaptarse a los retos técnicos o creativos que surgían durante el avance.

7.3. Adaptaciones durante el proceso

A lo largo del desarrollo se han realizado múltiples ajustes derivados de las pruebas o de ideas surgidas durante la implementación. Algunos ejemplos destacables son:

- El sistema de combate fue rediseñado para ser más modular y reutilizable, permitiendo aplicar daño a personajes y enemigos por igual.
- El personaje del Gato evolucionó desde una idea sencilla hasta incluir mecánicas complejas como escalada temporal y acceso a zonas de 1x1 bloque.
- La cámara fue adaptada para seguir dinámicamente al personaje seleccionado, ya que este se instancia en tiempo de ejecución.
- Se priorizó la estabilidad de los personajes existentes y se decidió posponer el Druida para una fase posterior.
- La implementación de enemigos como el fantasma o la planta lanza guisantes llevó a mejorar el sistema de detección, colisiones y estados.

Gracias a la estructura modular del código y al uso de control de versiones, estas adaptaciones no afectaron negativamente al flujo del proyecto. Al contrario, permitieron enriquecer las mecánicas y ofrecer una experiencia de juego más completa y coherente desde la base.

8. Planificación y Cronograma

8.1. Fases del desarrollo

La planificación del proyecto Velgrym se estructuró en seis fases principales, pensadas para cubrir desde la preparación del entorno hasta la entrega de una versión funcional y

documentada del juego. El desarrollo se llevó a cabo durante un período de 10 semanas aproximadamente, distribuidas de forma equilibrada para permitir iteración, pruebas y corrección de errores en cada etapa.

Las fases fueron:

- Fase 1 – Preparación del entorno
 - Instalación y configuración de Godot Engine.
 - Estructuración de carpetas, escenas y nodos base.
 - Pruebas iniciales de físicas y movimiento con personaje de prueba.
- Fase 2 – Desarrollo de personajes
 - Implementación completa del personaje Caballero (movimiento, salto, ataque).
 - Primer diseño y lógica básica del personaje Gato.
 - Creación del sistema modular de vida, daño y muerte.
- Fase 3 – Enemigos y combate
 - Implementación de enemigos básicos (slime, murciélago, seta).
 - Programación de IA simple (detección, persecución, patrullaje).
 - Integración con el sistema de combate modular.
- Fase 4 – Interfaz y HUD
 - Diseño de la barra de vida y pantalla de muerte.
 - Programación de la cámara para que siga al personaje instanciado.
 - Implementación de la selección de personaje y su carga en el mapa.
- Fase 5 – Diseño de niveles
 - Creación del mapa inicial (cueva) y del mapa principal (pradera).
 - Colocación de plataformas móviles, zonas elevadas y huecos estrechos.
 - Conexión entre escenas y puntos de aparición según personaje.
- Fase 6 – Pulido y documentación
 - Pruebas globales del sistema de movimiento, combate y enemigos.
 - Ajustes de físicas, colisiones, velocidad y animaciones.
 - Redacción y maquetación de la memoria técnica del proyecto.

8.2. Cronograma estimado

El siguiente cronograma muestra la distribución estimada del trabajo por semanas, equilibrando el avance técnico con el diseño jugable y visual:

Semana	Actividades principales	Objetivo
1–2	Preparación del entorno, físicas, estructura base del proyecto	Proyecto inicial funcional
3–4	Desarrollo del Caballero, sistema de vida y lógica de combate	Personaje principal jugable
5–6	Enemigos básicos, integración del sistema de combate, IA	Sistema de combate completo
7	Lógica y habilidades del Gato: escalada, agilidad, exploración	Segundo personaje operativo
8	HUD, cámara dinámica, sistema de muerte, selección de personajes	Interfaz funcional
9	Creación de niveles, conexión entre escenas, interacción con el entorno	Flujo de juego funcional
10	Testeo general, pulido de errores, documentación final	Versión preparada para presentación

Este cronograma ha servido como guía para distribuir las tareas sin sobrecargar semanas concretas, permitiendo trabajar con flexibilidad y adaptarse al avance real del proyecto.

8.3. Ajustes sobre la marcha

Durante el proceso de desarrollo se realizaron diversos ajustes para adaptarse a la evolución del proyecto y a la aparición de nuevas necesidades:

- El HUD necesitó reorganizarse para mantenerse funcional después de la muerte del personaje.
- El personaje Gato, inicialmente planteado como una opción secundaria, fue ampliado con mecánicas más complejas como la escalada temporal y el paso por huecos de 1x1 bloque.

- El sistema de combate se reprogramó para separar la lógica de daño, retroceso y animaciones, facilitando su reutilización tanto en enemigos como en personajes.
- La cámara fue rediseñada para seguir al personaje instanciado dinámicamente en lugar de un nodo estático.
- Se decidió posponer la implementación del personaje Druida para centrarse en la estabilidad y el pulido de los personajes ya jugables.
- Se añadieron enemigos adicionales como la planta lanza guisantes y el fantasma, enriqueciendo la variedad de amenazas en el mapa.

Gracias al enfoque iterativo y al uso de control de versiones, todos estos cambios se integraron de forma controlada sin comprometer la estructura general del proyecto ni alterar significativamente el calendario de trabajo. La planificación se mantuvo como una guía flexible, útil para adaptar el proyecto a las necesidades reales de implementación.

9. Desarrollo del proyecto

9.1. Preparación del entorno y organización inicial

Antes de comenzar con la implementación del videojuego, se realizó una fase inicial centrada en dejar preparado todo el entorno de desarrollo y la organización interna del proyecto.

Instalación del motor y herramientas

Se eligió Godot Engine por su carácter libre, su ligereza y su entorno visual basado en nodos. La versión utilizada fue la estable más reciente (Godot 3.5.2), totalmente compatible con los sistemas del entorno de trabajo.

Además, se instalaron:

- GIMP para retoques gráficos.
- Git y GitHub (repositorio privado) para control de versiones.

La configuración inicial permitió trabajar de forma modular y hacer seguimiento del proyecto desde el primer día.

Estructura de carpetas y organización del proyecto

Desde el principio se organizó el proyecto por carpetas temáticas para separar funcionalidades:

Velgrym/

```
|  
|— personajes/  
|— enemigos/  
|— mapas/  
|— hud/  
|— sistemas/  
|— recursos/  
| |— sprites/  
| |— sonidos/  
| |— tilesets/
```

Esta estructura ayudó a mantener un entorno limpio, comprensible y fácilmente escalable.

Escena base del juego

Se creó una escena vacía inicial Main.tscn que funciona como punto de entrada del juego. Desde ahí, se cargan dinámicamente el selector de personaje y posteriormente el mapa principal. Los primeros scripts globales creados fueron:

- Vida.gd: para gestionar salud y daño.
- CameraFollow.gd: para que la cámara siga al personaje instanciado.
- PlayerManager.gd: para guardar el personaje seleccionado y reutilizarlo.

Primera prueba de físicas y movimiento

La prueba inicial se hizo con un CharacterBody2D con código básico para verificar físicas:

```
extends CharacterBody2D
```

```
var velocidad = 100
```

```
var gravedad = 900
```

```
var salto = -300
```

```
var impulso = Vector2.ZERO
```

```
func _physics_process(delta):  
    impulso.y += gravedad * delta  
    if Input.is_action_pressed("mover_derecha"):  
        impulso.x = velocidad  
    elif Input.is_action_pressed("mover_izquierda"):  
        impulso.x = -velocidad  
    else:  
        impulso.x = 0  
  
    if is_on_floor() and Input.is_action_just_pressed("saltar"):  
        impulso.y = salto  
  
    move_and_slide()
```

Esta base sirvió para validar:

- Que la gravedad afectaba de forma natural.
- Que el salto respondía correctamente.
- Que se podía controlar la dirección y velocidad.

Una vez comprobado esto, se procedió a construir el sistema de personajes de forma más completa.

9.2. Sistema de movimiento y físicas básicas

Uno de los pilares fundamentales en el desarrollo de **Velgrym** fue la creación de un sistema de movimiento sólido, fluido y adaptable a distintos personajes. En un metroidvania, el control preciso y la respuesta del personaje marcan la diferencia entre una jugabilidad satisfactoria y una frustrante.

Desde el inicio, el sistema fue planteado como modular y configurable desde el editor, utilizando @export variables para facilitar ajustes sin necesidad de reescribir código.

Diseño inicial del movimiento

El movimiento se diseñó partiendo de un script base aplicado sobre un CharacterBody2D. El objetivo era permitir:

- Movimiento lateral con frenado progresivo.
- Salto con impulso vertical ajustable.
- Caída con gravedad acelerada.
- Detección precisa de suelo y paredes.

Fragmento de código base (MovimientoBase.gd):

```
extends CharacterBody2D
```

```
@export var velocidad := 120
```

```
@export var fuerza_salto := -350
```

```
@export var gravedad := 900
```

```
@export var friccion := 0.1
```

```
var impulso := Vector2.ZERO
```

```
func _physics_process(delta):
```

```
    impulso.y += gravedad * delta
```

```
    var direccion := Input.get_action_strength("mover_derecha") -  
Input.get_action_strength("mover_izquierda")
```

```
    impulso.x = lerp(impulso.x, direccion * velocidad, friccion)
```

```
    if is_on_floor() and Input.is_action_just_pressed("saltar"):
```

```
        impulso.y = fuerza_salto
```

```
    move_and_slide()
```

Ajustes por personaje

Cada personaje tenía diferentes sensaciones que transmitir:

- El **Caballero** debía sentirse más pesado, con menor velocidad, pero más fuerza.
- El **Gato** necesitaba agilidad, respuesta inmediata y capacidad para moverse rápidamente por zonas estrechas.

Esto se logró instanciando el script base en MovimientoCaballero.gd y MovimientoGato.gd, ajustando sus @export valores directamente desde el editor.

Detección de suelo y control de salto

Para garantizar precisión al detectar si el personaje estaba en el suelo, se utilizó `is_on_floor()` junto con `RayCast2D` en versiones posteriores para detectar suelos inclinados o plataformas móviles.

Se ajustó también la curva de salto, calibrando fuerza y gravedad para que el personaje no tuviera un salto flotante:

```
@export var max_velocidad_caida := 500
```

```
func _physics_process(delta):
```

```
    impulso.y += gravedad * delta
```

```
    if impulso.y > max_velocidad_caida:
```

```
        impulso.y = max_velocidad_caida
```

Esto evitó que la caída se volviera incontrolable o demasiado lenta.

Animaciones sincronizadas

Para reflejar el estado del personaje, se controlaron las animaciones mediante condiciones en tiempo real:

```
func actualizar_animacion():
```

```
    if !is_on_floor():
```

```
if impulso.y < 0:
```

```
    $Anim.play("salto")
```

```
else:
```

```
    $Anim.play("caida")
```

```
elif impulso.x == 0:
```

```
    $Anim.play("reposo")
```

```
else:
```

```
    $Anim.play("correr")
```

Esto garantizaba que la animación fuera coherente con el estado físico del personaje en todo momento.

Problemas detectados y soluciones

- **Resbalones en pendientes:** solucionado ajustando el parámetro `floor_max_angle`.
- **Sensación flotante al saltar:** corregido recalibrando la gravedad y la fuerza de salto.
- **Comportamiento idéntico en ambos personajes:** solucionado refactorizando el script y exponiendo variables individuales.

Ampliación del sistema con nuevas mecánicas

A partir del sistema base, se añadieron otras funcionalidades como:

- **Escalada temporal** (solo Gato), con detección lateral y temporizador de adherencia.
- **Salto desde pared**, con breve tiempo de pegado y rebote en dirección contraria.
- **Doble salto**, planificado, pero no implementado aún.

Ejemplo de detección de escalada (`MovimientoGato.gd`):

```
var escalando := false
```

```
var tiempo_escalada := 0.0
```

```
@export var max_escalada := 1.5 # segundos
```

```
func _physics_process(delta):
```

```
    if is_on_wall() and !is_on_floor():
```

```
        escalando = true
```

```
        tiempo_escalada += delta
```

```
        if tiempo_escalada > max_escalada:
```

```
            escalando = false
```

```
    else:
```

```
        escalando = false
```

```
        tiempo_escalada = 0.0
```

Conclusión del sistema

El sistema de movimiento resultante es:

- **Modular:** fácilmente adaptable a nuevos personajes o valores.
- **Responsivo:** ofrece control fluido y coherente.
- **Expandible:** puede crecer con nuevas habilidades (doble salto, dash, etc.).
- **Mantenible:** gracias a su separación lógica y uso de variables exportables.

Este sistema ha sido fundamental para establecer la base de la jugabilidad de Velgrym, asegurando que cada personaje se sienta diferente y que el movimiento sea parte activa de la experiencia.

9.3. Personajes jugables: implementación modular

Uno de los elementos clave en el diseño de Velgrym es la posibilidad de jugar con personajes distintos, cada uno con habilidades y estilos de juego únicos. Esta variedad no solo aporta rejugabilidad, sino que también está integrada en el diseño del mapa, ofreciendo rutas alternativas y ventajas específicas según el personaje elegido.

Selección de personaje y carga dinámica

Al iniciar la partida, el jugador elige entre dos personajes disponibles: el **Caballero** y el **Gato**. Esta selección se realiza en una escena de cueva inicial que contiene botones para seleccionar personaje.

Una vez elegido, el personaje se instancia de forma dinámica en la siguiente escena (la pradera). Para ello, se creó un autoload (JugadorManager.gd) que almacena la elección del jugador:

```
# JugadorManager.gd
var personaje_seleccionado = null

func set_personaje(ruta):
    personaje_seleccionado = ruta

func instanciar_personaje():
    var recurso = load(personaje_seleccionado)
    return recurso.instantiate()
```

De esta forma, el personaje se carga automáticamente en el mapa sin necesidad de duplicar escenas o scripts.

Diseño individual de personajes

Cada personaje tiene su propia escena y script derivados de una lógica común. Las escenas contienen nodos específicos (animaciones, colisiones, etc.) y sus propios valores de movimiento y habilidades.

Caballero: personaje estándar con movilidad media y combate cuerpo a cuerpo. Utiliza una maza pesada que afecta su impulso al atacar, provocando retroceso.

Gato: personaje ágil, no combate directamente pero puede escalar temporalmente, esquivar ataques y acceder a huecos estrechos. Tiene menos masa y mayor velocidad.

Druida (planificado): personaje de apoyo con ataques mágicos a distancia, curación y habilidades de manipulación ambiental. Su implementación se reserva para una versión futura.

Separación entre lógica común y específica

Para evitar duplicación de código, se creó un sistema de herencia. La clase base (MovimientoBase.gd) contiene funciones generales como movimiento, salto, y detección de suelo. Los scripts específicos como MovimientoCaballero.gd o MovimientoGato.gd heredan esta clase y sobrescriben o extienden funcionalidades:

```
# MovimientoCaballero.gd
extends "res://scripts/MovimientoBase.gd"

func atacar():
    if puede_atacar:
        $Anim.play("ataque")
        area_golpe.visible = true
gdscript
CopiarEditar
# MovimientoGato.gd
extends "res://scripts/MovimientoBase.gd"

func detectar_escalada():
    if is_on_wall() and !is_on_floor():
        escalando = true
```

Esto permitió mantener un sistema organizado y escalable, reduciendo errores y facilitando futuras ampliaciones.

Gestión de animaciones

Cada personaje tiene su propio conjunto de animaciones. El Caballero incluye animaciones de ataque, impacto, muerte y carrera pesada. El Gato tiene animaciones de carrera rápida, escalada, caída y sigilo.

Las animaciones se gestionan mediante un sistema de estados:

```
func actualizar_animacion():  
    if estado == "atacando":  
        $Anim.play("ataque")  
    elif is_on_floor() and impulso.x != 0:  
        $Anim.play("correr")  
    elif !is_on_floor() and impulso.y > 0:  
        $Anim.play("caida")
```

Esto garantiza que cada acción se represente visualmente de forma coherente, reforzando la sensación de control y diferenciación entre personajes.

Interacción única con el entorno

Cada personaje tiene formas distintas de relacionarse con el mapa:

- El **Caballero** puede golpear ciertos objetos para romperlos, desbloqueando rutas.
- El **Gato** puede entrar por huecos de 1x1 bloque y escalar temporalmente paredes.
- El **Druida** (futuro) podrá activar mecanismos mágicos y acceder a zonas mediante habilidades especiales.

Estas diferencias permiten que el diseño de niveles ofrezca caminos exclusivos y fomente la exploración según el personaje utilizado.

Pruebas y ajustes de equilibrio

Durante las pruebas, se ajustaron parámetros como la velocidad, la gravedad, el impulso del salto y la duración de habilidades. Se buscó que ningún personaje tuviera ventaja absoluta, sino que cada uno destacara en situaciones concretas.

Por ejemplo, el Caballero puede eliminar enemigos más fácilmente, pero no accede a zonas elevadas sin rutas alternativas. El Gato, en cambio, evita muchos combates y descubre rutas ocultas, pero no puede atacar directamente.

Escalabilidad y futuras ampliaciones

Gracias a la estructura modular y heredada, es posible añadir nuevos personajes sin alterar la lógica existente. Solo es necesario crear una nueva escena, derivar el script desde la clase base, asignar animaciones y registrar su ruta en el selector de personajes.

Esto permite incluir personajes desbloqueables, secretos o descargables en el futuro sin modificar la estructura principal del juego.

9.4. Sistema de combate, vida y muerte

El sistema de combate de Velgrym es una parte fundamental de la jugabilidad, ya que determina cómo interactúan los personajes con los enemigos, cómo se produce el daño y cómo se representa la muerte en pantalla. Desde su concepción, se diseñó como un sistema modular, reutilizable y fácilmente escalable, con el objetivo de permitir su uso tanto en los personajes jugables como en cualquier enemigo del juego.

El sistema está dividido en tres grandes bloques:

1. Mecánica de ataque.
2. Gestión de vida y daño.
3. Comportamiento al morir.

Mecánica de ataque

El ataque fue desarrollado inicialmente para el Caballero, el único personaje ofensivo implementado en esta versión. Se planteó como una acción limitada por un tiempo de reutilización (cooldown) y ligada a una animación concreta.

El área de impacto se implementó con un nodo Area2D, que se activa solamente durante ciertos frames de la animación de ataque para sincronizar perfectamente el golpe visual con el daño real.

Ejemplo de código del ataque:

```
func atacar():  
    if puede_atacar:  
        puede_atacar = false  
        $Anim.play("ataque")  
        $AreaGolpe.monitoring = true  
        yield($Anim, "animation_finished")  
        $AreaGolpe.monitoring = false  
        yield(get_tree().create_timer(0.5), "timeout")  
        puede_atacar = true
```

Esto garantiza que no se pueda atacar continuamente sin pausa, y que el impacto solo sea válido mientras el área está activa.

Gestión de vida y daño

Se creó un script independiente llamado Vida.gd, que puede asignarse a cualquier personaje o enemigo. Este script gestiona los puntos de vida, el estado de vulnerabilidad y la muerte.

Variables principales:

```
@export var vida_maxima := 5  
var vida_actual := vida_maxima  
var invulnerable := false
```

Función para recibir daño:

```
func recibir_danio(cantidad):  
    if not invulnerable:  
        vida_actual -= cantidad  
        invulnerable = true  
        $Anim.play("dano")  
        parpadear()  
        if vida_actual <= 0:  
            morir()  
        else:  
            yield(get_tree().create_timer(0.8), "timeout")  
            invulnerable = false
```

Este sistema se adapta automáticamente a enemigos o personajes, y permite aplicar retroceso, parpadeo y animaciones específicas tras recibir daño.

Retroceso al recibir daño

Al recibir daño, el personaje o enemigo es desplazado ligeramente en dirección contraria. Esto se implementó modificando el vector de impulso, añadiendo una fuerza momentánea hacia atrás.

```
func aplicar_retroceso(direccion):  
    impulso.x = direccion * retroceso_fuerza
```

Este efecto mejora la respuesta visual del combate y añade una capa estratégica, ya que permite alejar al enemigo momentáneamente tras golpearlo.

Muerte y limpieza

Cuando la vida llega a cero, se desactiva la lógica del nodo y se reproduce una animación de muerte. En el caso de enemigos, se elimina tras unos segundos. En el caso del jugador, se muestra una pantalla de muerte con opción de reinicio.

Función de muerte general:

```
func morir():  
    set_process(false)
```

```
$Anim.play("muerte")  
yield($Anim, "animation_finished")  
queue_free()
```

En el caso del jugador, se activa un menú de “game over” y se reestablece el personaje en el último punto seguro.

Invulnerabilidad temporal

Después de recibir daño, el personaje entra en un estado de invulnerabilidad durante 0.8 segundos, para evitar recibir golpes múltiples de forma injusta. Durante ese tiempo, el sprite parpadea como señal visual.

Función de parpadeo:

```
func parpadear():  
    for i in range(6):  
        visible = not visible  
        yield(get_tree().create_timer(0.1), "timeout")  
    visible = true
```

Este mismo sistema puede aplicarse a enemigos fuertes o jefes en el futuro para evitar ataques en cadena.

Integración con el HUD

El HUD incluye una barra de vida que se actualiza automáticamente. Esta barra escucha la señal `vida_actualizada` emitida desde el script de vida y ajusta su escala o número de corazones según corresponda.

Problemas y soluciones

Durante el desarrollo del sistema de combate se enfrentaron varios problemas:

- La colisión del ataque no coincidía con la animación. Se solucionó activando el área solo durante los frames adecuados.
- El retroceso no funcionaba correctamente si el personaje estaba en el aire. Se aplicó una lógica diferente para suelo y aire.
- Algunos enemigos seguían ejecutando su lógica tras morir. Se añadió `set_process(false)` y eliminación controlada con `queue_free()`.

Escalabilidad del sistema

El sistema está preparado para futuras mejoras como:

- Estados alterados (veneno, fuego, congelación).
- Enemigos con defensa parcial o armaduras.
- Jefes con múltiples fases.
- Personajes con ataques a distancia (como el Druida).

Al ser modular, solo es necesario añadir lógica propia a enemigos específicos, reutilizando la base común de vida y daño.

En resumen, el sistema de combate de Velgrym combina simplicidad técnica con suficiente profundidad mecánica. Está diseñado para ser visualmente claro, mecánicamente justo y fácilmente ampliable, cumpliendo con los requisitos esenciales para un juego tipo metroidvania.

9.5. Inteligencia artificial y comportamiento de enemigos

En Velgrym, los enemigos no solo representan un obstáculo mecánico, sino que refuerzan la ambientación, el diseño del mapa y la estrategia del jugador. Se ha implementado un sistema

de inteligencia artificial (IA) básica con estructuras modulares, que permite definir distintos comportamientos sin duplicar lógica.

Estructura común

Todos los enemigos comparten una estructura base que incluye:

- Un nodo de colisión para detectar al jugador.
- Un sistema de vida reutilizable.
- Animaciones específicas según estado.
- Un script con lógica de comportamiento.

Esto permite que cualquier nuevo enemigo pueda implementarse rápidamente combinando estos elementos y ajustando solo la lógica particular.

Comportamientos implementados

Durante esta fase se han desarrollado cinco enemigos distintos:

Slime

Se desplaza lentamente por el suelo y, al detectar al jugador, lo persigue. Puede adherirse temporalmente a paredes, cambiando su gravedad y orientación. Para ello, se modifica el eje de gravedad y se usan colisiones laterales.

Murciélago

Permanece colgado del techo hasta que el jugador entra en su rango. Entonces se lanza en picado. Si falla el ataque, regresa a su posición original. Su IA gestiona dos estados: reposo y ataque.

Seta

Camina de un lado a otro sin detectar al jugador. Solo puede ser destruida si el jugador cae encima. Este enemigo introduce una mecánica diferente que fuerza el uso del entorno y fomenta la movilidad vertical.

Fantasma

Se mueve lentamente hacia el jugador y atraviesa paredes. No puede ser dañado con ataques normales. Sirve como presión constante y obliga a moverse con rapidez.

Planta lanza guisantes

Permanece estática y dispara proyectiles a intervalos regulares. Utiliza temporizadores y nodos de dirección para instanciar las balas.

IA basada en detección de cuerpos

Se emplean nodos Area2D con colisiones para detectar cuándo el jugador entra en el rango de acción del enemigo. Al activarse, se cambia el estado del enemigo de pasivo a agresivo. Si el jugador sale del área, algunos enemigos retornan a su estado inicial.

Ejemplo de lógica básica

El siguiente fragmento describe un comportamiento genérico:

```
if estado == "reposo" and jugador_en_area:
    estado = "perseguir"

if estado == "perseguir":
    seguir_al_jugador()
    if distancia_al_jugador < distancia_ataque:
        estado = "atacar"
```

Este sistema puede adaptarse fácilmente a otros comportamientos como patrullaje, huida o invocación de aliados.

Gestión de estados

Cada enemigo puede tener varios estados:

- Reposo
- Patrullaje
- Persecución
- Ataque

- Retorno a posición
- Muerte

La transición entre estados se realiza mediante condiciones evaluadas en cada ciclo de `_process` o `_physics_process`.

Animaciones y efectos visuales

Las animaciones reflejan claramente el estado del enemigo. Por ejemplo, el murciélago reproduce una animación de vuelo rápido al atacar y vuelve a la animación de reposo al regresar al techo. Se han añadido efectos visuales simples, como parpadeo al recibir daño y rotación o disolución al morir.

Problemas encontrados

Durante el desarrollo se detectaron algunos errores:

- Enemigos que seguían atacando tras morir: se corrigió desactivando su lógica tras llegar a 0 de vida.
- Detección errónea en plataformas elevadas: se ajustaron las áreas de colisión para diferenciar altura.
- Bajadas de rendimiento al tener varios enemigos activos: se implementó una desactivación automática para enemigos fuera del campo de visión.

Escalabilidad

La estructura modular permite crear nuevos enemigos fácilmente, cambiando solo la lógica interna. También permite añadir jefes, enemigos con varias fases o invocadores. Ya están previstas mejoras como enemigos con patrones de patrullaje más complejos, emboscadas, ataques a distancia inteligentes o resistencias específicas al daño.

En resumen, la inteligencia artificial de los enemigos en Velgrym se basa en un sistema sencillo, claro y reutilizable, que ha permitido implementar una buena variedad de comportamientos sin complicar el código ni comprometer el rendimiento. Está preparada para ampliarse en futuras fases del desarrollo.

9.6. Creación de mapas y elementos interactivos

El diseño de mapas en Velgrym cumple una doble función: por un lado, define el recorrido visual y estético del juego; por otro, condiciona directamente la experiencia jugable a través de la exploración, el combate y el uso de habilidades específicas según el personaje.

Durante el desarrollo se han creado dos mapas principales completamente funcionales: la **cueva inicial** y la **pradera**. Cada uno cumple un propósito distinto dentro del flujo de juego.

Mapa 1 – Cueva de inicio

La cueva es la primera escena que ve el jugador. Su objetivo no es presentar un reto, sino permitir la **selección de personaje**. Mediante botones en pantalla, el jugador puede elegir jugar como Caballero o como Gato.

Una vez realizada la selección, se guarda la ruta del personaje en una variable de un autoload (JugadorManager.gd) y se carga el siguiente mapa (la pradera), instanciando el personaje correspondiente.

Este mapa no contiene enemigos ni elementos interactivos, pero cumple una función estructural esencial: gestionar la lógica de selección de personaje sin duplicar escenas posteriores.

Mapa 2 – Pradera

La pradera es el primer nivel real del juego. Es un entorno natural de tamaño medio, compuesto por plataformas, caminos verticales y zonas con múltiples rutas posibles.

El mapa está diseñado para introducir progresivamente al jugador en las principales mecánicas del juego: movimiento, combate, enemigos y exploración. La colocación de enemigos, plataformas y zonas bloqueadas varía según el tipo de personaje que se haya elegido.

Por ejemplo:

- El Caballero debe avanzar enfrentando a los enemigos directamente.
- El Gato puede evitar combates utilizando rutas elevadas, huecos estrechos o escalada temporal.

Elementos interactivos implementados

Durante esta fase se han añadido los siguientes elementos reales al mapa:

- **Plataformas móviles:** se desplazan en bucle horizontal o vertical. Requieren saltos sincronizados para alcanzar zonas elevadas o cruzar vacíos.
- **Huecos estrechos (1x1 bloque):** permiten el paso únicamente al Gato. Están colocados de forma estratégica para acceder a atajos o zonas ocultas.
- **Distribución de enemigos:** slimes, murciélagos, setas, fantasmas y plantas están posicionados con intención. El jugador debe decidir si enfrentarlos, evitarlos o usar el entorno a su favor.
- **Zonas de caída y salto vertical:** la estructura vertical del mapa permite aproximarse a las zonas desde arriba o desde el lateral, fomentando la exploración y la toma de decisiones.

Compatibilidad con las habilidades de cada personaje

El diseño del mapa está adaptado a las capacidades únicas de los personajes jugables:

- El Caballero no puede entrar en huecos pequeños ni escalar, pero puede romper obstáculos (planificado) y tiene más resistencia en combate.
- El Gato puede evitar enemigos, acceder a zonas exclusivas y moverse con mayor libertad, pero no puede atacar directamente.

Esto refuerza el enfoque metroidvania del juego, en el que cada personaje no solo combate de forma distinta, sino que también se relaciona de forma distinta con el mundo.

Elementos planificados, pero no implementados

Actualmente, algunos elementos están diseñados, pero no presentes en esta versión:

- **Puertas mágicas.**

- **Rocas rompibles.**
- **Trampas activas** como pinchos, fuego o proyectiles ocultos.

El mapa está estructurado para poder integrarlos fácilmente en el futuro sin necesidad de rehacer el diseño actual.

Problemas detectados y soluciones aplicadas

Durante el diseño de niveles se enfrentaron algunos errores técnicos que fueron resueltos:

- **La cámara no seguía al personaje instanciado:** se solucionó enlazando dinámicamente la cámara al nodo del personaje cargado desde la cueva.
- **El HUD no se mostraba correctamente tras la muerte:** se ajustó su posición en la jerarquía y se forzó a seguir a la cámara.
- **El personaje aparecía en lugares incorrectos:** se utilizó un nodo Spawn como punto de entrada, con control de aparición según el personaje.
- **Errores al cambiar entre personajes:** se centralizó la lógica de carga de personaje y se asignaron referencias comunes al HUD, cámara y sistema de vida.

Conclusión

Los dos mapas actuales —cueva y pradera— cumplen con éxito sus objetivos iniciales: permiten probar personajes, explorar el entorno, combatir enemigos y descubrir rutas alternativas. Están diseñados con una estructura flexible, que permite su ampliación con nuevas zonas, enemigos o mecánicas en futuras versiones sin comprometer la estabilidad del sistema actual.

9.7. HUD, cámara y transiciones de escenas

El sistema de interfaz gráfica (HUD), la cámara y las transiciones entre escenas son elementos clave en Velgrym para garantizar una experiencia fluida y coherente. Todos estos sistemas han sido implementados con un enfoque funcional, adaptado al flujo del juego y al sistema de personajes seleccionables.

HUD: barra de vida y sistema de muerte

El HUD actual se compone de una barra de vida que se muestra en pantalla durante la partida. Esta barra se actualiza de forma automática cuando el jugador recibe daño, reflejando visualmente su estado de salud.

También se ha implementado una pantalla de muerte básica que aparece cuando la vida del personaje llega a cero. En esta versión, dicha pantalla detiene momentáneamente el juego y ofrece la posibilidad de reiniciar la escena, permitiendo volver a jugar desde el inicio del nivel. El HUD se ha configurado para reaparecer correctamente tras la muerte y reinicio del personaje.

Cámara dinámica

La cámara ha sido configurada para seguir automáticamente al personaje activo, ya que este no está presente en el mapa hasta que el jugador lo selecciona. Esto supuso un reto inicial, ya que la cámara debía vincularse en tiempo de ejecución al personaje instanciado.

Se resolvió haciendo que cada personaje tenga su propia cámara, la cual se activa automáticamente al cargar el mapa correspondiente. El movimiento de la cámara incluye un efecto de suavizado, lo que mejora la sensación de fluidez durante el desplazamiento lateral.

Transiciones entre escenas

Las transiciones entre escenas están completamente funcionales. El juego comienza en la cueva de selección de personajes, y tras elegir uno, se carga el mapa principal (la pradera) con el personaje ya activo y preparado para jugar.

Este sistema de transición es simple, estable y evita duplicaciones innecesarias de lógica. El HUD, la cámara y la lógica de combate se activan de forma correcta tras la transición, asegurando una experiencia continua para el jugador.

Preparado para expansión

Aunque actualmente el sistema se limita a dos escenas jugables, está preparado para escalar a:

- Nuevos mapas conectados entre sí.

- Puntos de guardado y reaparición.
- Menús más complejos o animaciones de transición.

Conclusión

El sistema de HUD, cámara y transiciones en Velgrym funciona correctamente y ha demostrado ser estable en pruebas. Su diseño modular permite su ampliación sin necesidad de rehacer las escenas existentes, lo que facilita el crecimiento del proyecto en futuras versiones.

9.8. Pruebas, ajustes y resolución de errores

Durante todo el desarrollo de Velgrym se llevaron a cabo pruebas constantes para validar cada sistema implementado: movimiento, combate, enemigos, HUD, cámara y transición de escenas. Estas pruebas fueron fundamentales para detectar errores, pulir el comportamiento de los personajes y garantizar una experiencia de juego coherente y estable.

Pruebas de movimiento y físicas

Desde las primeras versiones, se probó a fondo el sistema de movimiento para asegurar que cada personaje se sintiera diferente y responsivo. Se ajustaron valores como velocidad, fuerza de salto, gravedad y fricción hasta conseguir que el control del personaje fuera fluido y preciso.

Uno de los principales problemas detectados fue la sensación "flotante" al saltar, que se corrigió aumentando la gravedad y ajustando la curva de salto. También se solucionaron resbalones en superficies inclinadas ajustando la fricción y el comportamiento de colisión con el suelo.

Pruebas de combate y retroalimentación

Durante el combate, se probaron múltiples combinaciones de enemigos y situaciones. Se corrigieron errores como:

- Enemigos que seguían atacando tras morir.
- Colisiones de ataques que no coincidían con la animación.

- Retroceso que aplicaba fuerzas incorrectas si el personaje estaba en el aire o cerca de una pared.

Además, se ajustó la duración de la invulnerabilidad tras recibir daño, y se mejoró la visibilidad del impacto mediante parpadeo del sprite y efectos visuales sencillos.

Pruebas del sistema de enemigos

Se probó el comportamiento individual de todos los enemigos implementados (slime, murciélago, seta, fantasma, planta). Se realizaron ajustes en sus áreas de detección, patrones de movimiento, velocidad y puntos de aparición.

También se verificó que los enemigos se comportaran correctamente según el personaje seleccionado, y que el jugador tuviera alternativas según su estilo de juego (enfrentamiento directo con el Caballero o evasión con el Gato).

Problemas con la cámara y el HUD

Uno de los errores más comunes fue que la cámara no seguía correctamente al personaje instanciado al cambiar de escena. Se solucionó enlazando la cámara al personaje en tiempo de ejecución, asegurando que cada personaje tuviera su propia cámara correctamente configurada.

El HUD también presentó errores de visualización tras la muerte del personaje. Para solucionarlo, se reorganizó la jerarquía de nodos y se hizo que el HUD se centrara en la cámara, no en el personaje.

Plataformas y colisiones

En las primeras pruebas, el personaje podía quedarse atrapado en plataformas móviles o ser empujado al borde de la pantalla. Esto se solucionó ajustando los márgenes de colisión y los tiempos de movimiento de las plataformas.

Ajustes de dificultad y equilibrio

Se dedicó tiempo a equilibrar la velocidad de los enemigos, su daño, y el comportamiento según el personaje. Por ejemplo, se redujo la agresividad del fantasma para que no persiguiera indefinidamente, y se ajustó la velocidad de las plantas lanzaguisantes para que dieran tiempo de reacción.

También se probaron rutas específicas del Gato y se aseguraron zonas seguras donde pudiera escalar o evitar combates.

Conclusión

El proceso de pruebas y ajustes ha sido esencial para consolidar una versión jugable y estable de Velgrym. Gracias a estas iteraciones, se logró corregir errores críticos, mejorar la jugabilidad general y preparar una base sólida sobre la que seguir ampliando el juego en futuras fases.

9.9. Documentación y control de calidad final

Desde el inicio del desarrollo de Velgrym se ha mantenido una documentación constante y ordenada. Este documento forma parte de esa planificación, y recoge todas las fases del proyecto: desde la preparación del entorno, pasando por la implementación técnica, hasta los ajustes finales antes de la entrega.

Documentación técnica

La memoria se ha elaborado en paralelo al desarrollo, lo que ha permitido reflejar con precisión las decisiones tomadas, los problemas detectados y las soluciones aplicadas. Cada sistema importante (movimiento, combate, enemigos, HUD, mapas) ha sido descrito y estructurado para facilitar su comprensión, tanto para uso académico como para futuras ampliaciones del juego.

Además de esta memoria, también se ha incluido documentación interna en el proyecto:

- Comentarios explicativos dentro del código, especialmente en scripts reutilizables como el sistema de vida o el controlador de movimiento.

- Organización clara de carpetas en el árbol de archivos de Godot, separando personajes, enemigos, escenas, sistemas, recursos visuales y de sonido.
- Archivos con versiones de prueba y anotaciones que acompañan los principales cambios implementados.

Control de versiones

Se ha utilizado Git como sistema de control de versiones, alojando el proyecto en un repositorio privado en GitHub. Aunque no se ha trabajado con múltiples ramas, el uso de una rama principal y *commit* descriptivos ha permitido hacer seguimiento del progreso, recuperar versiones estables y corregir errores de forma organizada.

Este control ha sido clave para mantener la estabilidad del proyecto durante los cambios más profundos, como la implementación modular de personajes o la reestructuración del sistema de combate.

Verificaciones de estabilidad

Antes de finalizar el desarrollo, se realizaron pruebas completas de principio a fin con los dos personajes disponibles: Caballero y Gato. En ambas rutas se verificó que funcionaran correctamente:

- El movimiento y salto.
- Las interacciones con enemigos.
- La activación del HUD.
- El seguimiento de la cámara.
- La aparición correcta tras la selección del personaje.
- La muerte y reinicio de la escena sin errores.

Se comprobó que los enemigos se comportaran como se esperaba y que los elementos interactivos del mapa respondieran adecuadamente en cada caso. También se revisó el equilibrio entre personajes, el rendimiento general y la coherencia de las animaciones.

Conclusión

El control de calidad final ha confirmado que el proyecto cumple con los objetivos propuestos para esta fase. Se trata de una versión jugable, funcional y ampliable, desarrollada con criterios de modularidad, orden y estabilidad técnica.

Gracias al enfoque mantenido durante el desarrollo, Velgrym no solo presenta una base sólida, sino también una documentación clara y un entorno estructurado, listos para seguir creciendo en futuras versiones del juego.

10. Resultados

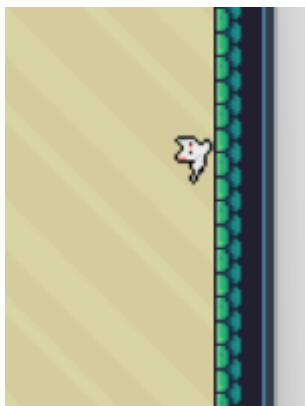
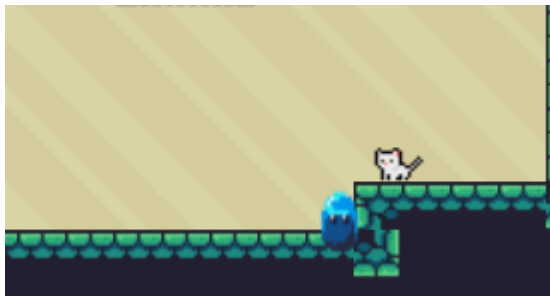
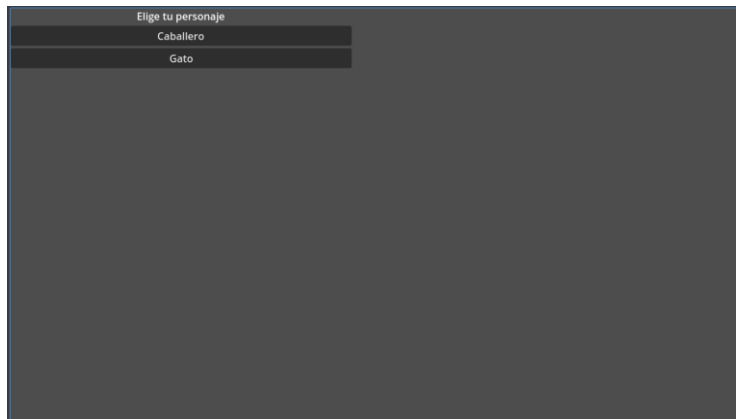
El desarrollo del videojuego Velgrym ha dado como resultado una versión funcional y estable, en la que se pueden probar todas las mecánicas implementadas hasta el momento. El jugador puede seleccionar entre dos personajes jugables (Caballero y Gato), cada uno con habilidades únicas, y comenzar la partida en un mapa interconectado con enemigos activos, plataformas móviles y rutas diferenciadas.

El sistema de movimiento, combate, HUD, enemigos y cámara ha sido completamente integrado y validado mediante pruebas reales de jugabilidad. Todos los elementos funcionan de forma coherente y sin errores críticos.

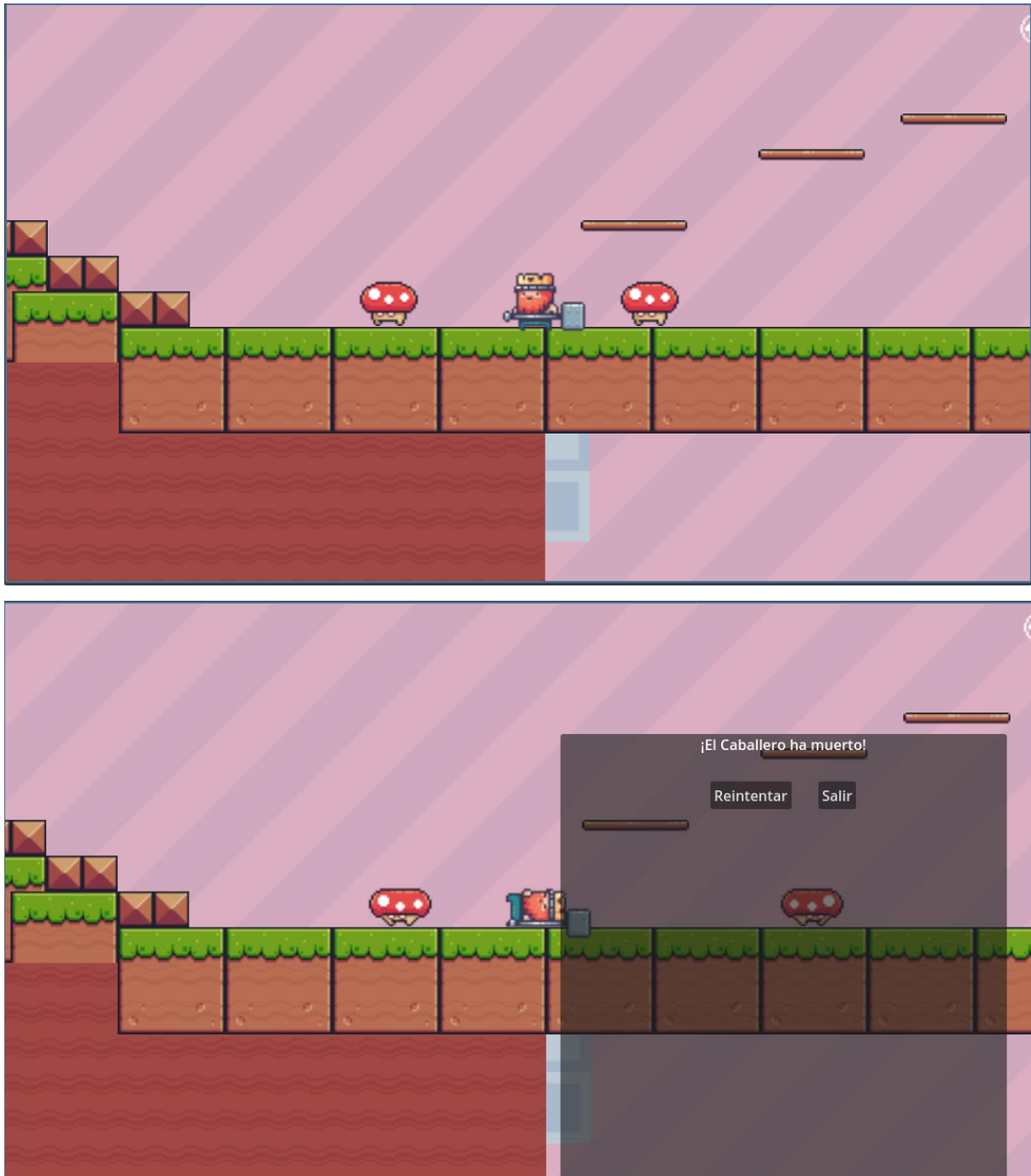
El proyecto completo se encuentra alojado en un repositorio privado de GitHub, accesible desde el siguiente enlace:

<https://github.com/JOSEANTONIOVZ123/Velgrym.git>

Además, se han capturado varias imágenes del juego en funcionamiento, tanto en la selección de personaje como en el mapa principal, enfrentándose a enemigos y utilizando las habilidades específicas de cada personaje.







Estas evidencias visuales sirven para demostrar el funcionamiento real de la aplicación y validar los objetivos técnicos alcanzados.

11. Conclusiones y Futuras Mejoras

11.1. Valoración personal del proyecto

El desarrollo de Velgrym ha supuesto un reto técnico y creativo completo que ha permitido aplicar de forma práctica los conocimientos adquiridos a lo largo del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma.

Durante el proceso se ha trabajado con múltiples áreas del desarrollo de software, como la lógica de programación, el diseño de físicas personalizadas, la inteligencia artificial básica para enemigos, los sistemas de vida y combate modulares, el manejo de escenas dinámicas, el control de cámara y el diseño estructurado de mapas jugables. También se ha trabajado en el diseño visual y la integración de interfaz gráfica (HUD), lo que ha permitido una aproximación realista a un flujo de desarrollo completo.

El uso del motor Godot ha sido clave, tanto por su flexibilidad como por su organización basada en nodos y escenas reutilizables. Esta arquitectura ha facilitado el trabajo modular, la escalabilidad y la integración progresiva de nuevos elementos sin necesidad de reestructurar continuamente el proyecto.

La evolución del juego, desde una idea inicial basada en un metroidvania con varios personajes, hasta una versión funcional con personajes diferenciados, enemigos activos, un mapa exploratorio y un sistema de combate completo, demuestra que es posible crear un videojuego con identidad propia en un entorno individual, gestionado con herramientas reales como GitHub y aplicando una metodología ágil adaptada al trabajo en solitario.

Además de la parte técnica, el proyecto ha servido como una oportunidad para desarrollar competencias en organización, priorización de tareas, resolución de problemas y documentación continua. En conjunto, ha sido una experiencia completa y formativa que refleja fielmente los objetivos del módulo de desarrollo de aplicaciones.

11.2. Propuestas de mejora

Aunque el proyecto alcanza un estado jugable y estable, existen múltiples áreas que podrían mejorarse o ampliarse:

- **Incorporar el tercer personaje jugable (Druida)**, con habilidades de hechizos, curación y control del entorno, que completen la experiencia del trío principal.

- **Ampliar el número de mapas** y zonas conectadas, reforzando la estructura metroidvania con rutas más complejas, áreas secretas y múltiples puntos de entrada y salida.
- **Implementar un sistema de guardado y carga**, con puntos de control repartidos en el mapa y restauración del progreso.
- **Mejorar el HUD**, añadiendo indicadores de objetos recolectados, habilidades activas, progreso del nivel o energía mágica.
- **Crear nuevos enemigos** con comportamientos más variados, como enemigos a distancia, en grupo o con armadura que requiera golpes múltiples.
- **Diseñar jefes finales o minibosses**, con patrones de ataque únicos, animaciones especiales y escenarios diseñados para combates más elaborados.
- **Pulir el apartado visual y sonoro**, con más efectos, transiciones, animaciones específicas y música adaptada por zona.

11.3. Líneas futuras de desarrollo

De cara al futuro, Velgrym tiene el potencial de evolucionar hacia una versión más ambiciosa, ampliada y publicable en plataformas independientes, manteniendo su esencia jugable, pero explorando nuevos caminos creativos.

Algunas líneas de desarrollo previstas son:

- **Integrar la mecánica del mundo espejo**, una dimensión paralela con enemigos alternativos, rutas distintas y zonas inaccesibles desde el mundo real. Esta mecánica permitiría reutilizar mapas de forma creativa y añadir desafíos adicionales sin duplicar contenido.
- **Optimizar el rendimiento** para hacerlo compatible con dispositivos móviles o plataformas portátiles, ampliando su accesibilidad.
- **Añadir narrativa ligera**, con diálogos opcionales, objetos con descripciones y textos ocultos que refuercen el lore del mundo y permitan al jugador descubrir detalles del pasado sin romper el ritmo de juego.

- **Incluir un sistema de progreso desbloqueable**, que permita al jugador mejorar a los personajes, conseguir nuevas habilidades o acceder a contenidos extra mediante exploración o desafíos especiales.
- **Ampliar las opciones de accesibilidad y configuración**, incluyendo remapeo de controles, modo sin combate (solo exploración) o soporte parcial para mando.

Estas mejoras no solo harían crecer el juego en contenido, sino también en profundidad, permitiendo que Velgrym evolucione más allá del ámbito académico y se convierta en una experiencia completa para cualquier tipo de jugador.

12. Bibliografía

A continuación se recoge la relación de fuentes y recursos utilizados durante el desarrollo del proyecto Velgrym, tanto para el desarrollo técnico como para el apartado gráfico y sonoro:

Desarrollo con Godot y programación en GDScript

- Godot Engine – Documentación oficial. Disponible en: <https://docs.godotengine.org>
- Godot Recipes – Ejemplos prácticos y tutoriales para resolver problemas comunes. Disponible en: https://kidscancode.org/godot_recipes/
- GDQuest – Tutoriales y artículos sobre buenas prácticas en Godot. Disponible en: <https://www.gdquest.com>
- HeartBeast – Curso completo de desarrollo de videojuegos en Godot. Disponible en: <https://www.youtube.com/@HeartBeast>
- GitHub Docs – Guía de uso básico de Git y control de versiones. Disponible en: <https://docs.github.com>

Recursos gráficos y sonoros utilizados

- Kenney.nl – Recursos gratuitos de audio y efectos visuales. Disponible en: <https://kenney.nl/assets>
- Freesound.org – Efectos de sonido bajo licencia Creative Commons. Disponible en: <https://freesound.org>

- ToffeeCraft – Cat Pack (personaje Gato). Disponible en: <https://toffeecraft.itch.io/cat-pack>
- Game Endeavor – Mystic Woods (tilesets naturales y decorativos). Disponible en: <https://game-endeavor.itch.io/mystic-woods>
- PixelFrog – Treasure Hunters. Disponible en: <https://pixelfrog-assets.itch.io/treasure-hunters>
- PixelFrog – Pixel Adventure 1. Disponible en: <https://pixelfrog-assets.itch.io/pixel-adventure-1>
- CraftPix – Free Slime Sprite Sheets. Disponible en: <https://craftpix.net/freebies/free-slime-sprite-sheets-pixel-art/>

Herramientas gráficas

- GIMP – Editor de imágenes libre. Disponible en: <https://www.gimp.org>
- Aseprite – Editor de pixel art. Disponible en: <https://www.aseprite.org>
- Piskel – Editor online de sprites. Disponible en: <https://www.piskelapp.com>

13. Anexos

Anexo 1 – Repositorio del proyecto

<https://github.com/JOSEANTONIOVZ123/Velgrym.git>

En este repositorio esta TODO el proyecto de Velgrym donde puedes incluso ver el código fuente y los assets usados.